

Mechanics of Flexible Structures and Soft Robots

MAE-263F

Fall 2025

Luke Sage

Homework 1

Problem 1

Pseudocode and Code structure:

Read inputs:

Open text file “nodes.txt” and read out each node, convert the nodes to an array

Open text file “free_nodes.txt” and store a list of each of the non-constrained nodes

Open text file “springs.txt” and store an array of the indexes of pairs of nodes that springs connect, as well as the stiffnesses of each spring.

Preparation for main loop:

Calculate ndof, the number of degrees of freedom (Number of nodes * 2)

Initialize position, velocity, and acceleration vectors.

Calculate resting length of each spring, and store in vector l_k

Create vector w from node masses to store the weight force on each node.

Simulation loop setup:

Set time step ($\Delta t=0.01s$)

Set the max amount of time, then create a vector from 0 to max time to iterate over

Initialize vector indicating free degrees of freedom, and convert free nodes into degrees of freedom

Create a vector that stores the y-coordinate of a given node (or nodes) for plotting.

Plot the setup at $t=0$ before the loop begins

Main simulation loop:

Iterate over time steps until the max time is reached

Call my integrator to with old position and velocities to get new vectors

My integrator:

Commented [LS1]: Remember to cite where the original course material came from

Create a guess for a new x position and generate a tolerance value for the Newton Raphson method.

Run the Newton Raphson solver to generate a new position and velocity vector.

Call a function to generate force and Jacobian matrices based on positions

From these forces, only modify the position of free degrees of freedom

From new positions, calculate new velocities

If the new solution is within tolerance, return new position and velocities.

If the time step is a particular value, (0, 0.1, 1, 10, or 100s) output a plot showing the positions of the nodes and springs.

Set the calculated position and velocity vectors to the old position and velocity vectors

Final plot:

Plot y position of nodes of interest for the time interval of simulation.

Brief description of main functions and scripts:

Main

Inputs: 3 text files: nodes.txt, free_nodes.txt, and springs.txt

Outputs: Plots at selected time intervals of the node and spring network, plot of y-position of nodes of interest over time

Main is the script that calls the rest of the functions. It contains the main loop, as well as all the setup information required to call myInt.

MyInt

Inputs: t_new, x_old, u_old, free_DOF, stiffness_matrix, index_matrix, m, dt, l_k

Outputs, x_new, u_new

MyInt is the newton Raphson integrator. It creates an initial guess, then calls getForceJacobian to generate the forces and jacobian matrix based on the positions of each node and the stiffness of all of the springs, as well as the external forces. Based on

these outputs, it generates a new guess for the position matrix, as well as the velocities. It then compares to the error, and iterates until the tolerance is reached.

GetForceJacobian

Inputs: x_new , x_old , u_old , $stiffness_matrix$, $index_matrix$, m , dt , l_k

Outputs: f , J (force, jacobian)

GetForceJacobian calculates new forces and the jacobian of the network given new and old positions, and old velocities. It takes into account the stiffnesses of the springs as well as external forces. It calls gradEs, HessES, and getFexternal.

GradEs:

Inputs: x_k , y_k , x_{kp1} , y_{kp1} , l_k , k

Outputs: F

GradEs calculates the gradient of the stretching energy given the coordinates of the current iteration and the previous iteration. It returns F , which is the gradient array.

HessEs:

Inputs: x_k , y_k , x_{kp1} , y_{kp1} , l_k , k

Outputs: J

HessEs gives the 4x4 matrix Hessian of the spring energy. It calculates this from the current and previous positions.

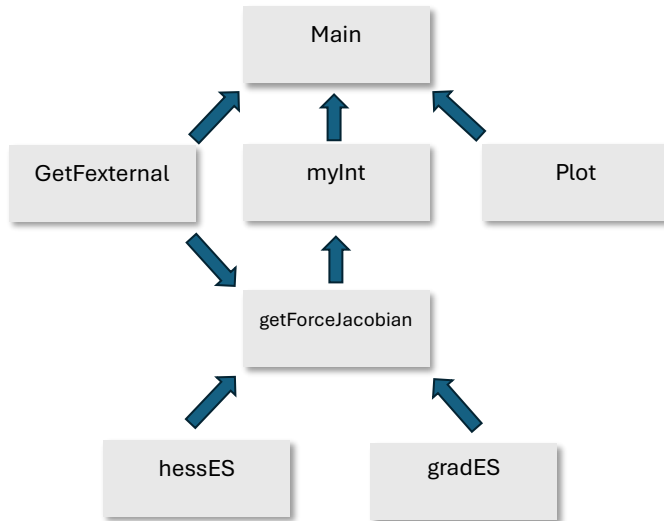
getFExternal

Inputs: m

Outputs: w

getFExternal takes in a mass value, and outputs a weight vector of the same size as the mass one.

Block Diagram



Problem 2

To choose an appropriate step, a time sensitivity study should be done.

For the purposes of this homework, an informal study was completed, but for a larger scope project, it would be useful to plot an output as a function of delta t. The output will likely be something like final rest position or terminal velocity that should be unaffected by node count. This will result in a plot showing, for small time steps, no significant relationship between delta t and the output. For larger time steps, more variation will be introduced, and the simulation might not converge. It is then possible to pick a time value roughly

For this project, a simple test was run, and the time step varied from .01 to 0.5 seconds. The simulation did not converge at time steps larger than 0.2. Values smaller than 0.2s largely held a similar final resting position of one of the nodes of roughly -1.08m. This indicates that likely, a reasonable value for the time step is a value less than 0.2. 0.01, the initial time step value, is likely more precise than is necessary.

Problem 3

Below is the code for the explicit Euler integrator:

```

def myIntExp(t_new, x_old, u_old, free_DOF, stiffness_matrix, index_matrix, m, dt, l_k):

    x_new = x_old + u_old * dt
    print("->", x_new, u_old)

    f, J = getForceJacobian(x_new, x_old, u_old, stiffness_matrix, index_matrix, m, dt, l_k)

    f_free = f[free_DOF]
    #print("f_free = ", f_free)

    u_new = u_old
    x_new = x_old

    u_new[free_DOF] = u_old[free_DOF] - (f[free_DOF] / m[free_DOF]) * dt
    x_new[free_DOF] = x_old[free_DOF] + u_new[free_DOF] * dt

    return x_new, u_new

```

A time step of 0.01s allows the simulation to run without issue, and results in a node position curve similar to that of the explicit simulator. Because of the relatively small processing time of this problem, the simplicity of the explicit Euler is an advantage.

I believe the method I ended up using is technically a Explicit Euler hybrid, as it establishes a x_{new} to input to the getForceJacobian such that the script does not require an additional force function.

Problem 4:

The Newmark Beta integrator is a method for solving a system of differential equations. The assumption the Newmark beta integrator makes is constant or linear acceleration.

By changing the parameters in the equations, we can move from the explicit central scheme with $\gamma = 0.5$ and $\beta = 0$ to the average constant acceleration scheme with $\gamma = 0.5$, $\beta = 0.25$. Generally, the damping is controlled via the gamma parameter. We are able to adjust the parameters to control the artificial damping, but also ensure that we don't diverge.

Generally, $\gamma = 0.5$ is regarded as a value that will avoid artificial damping.